A 2.92-Gb/s/W and 0.43-Gb/s/MG Flexible and Scalable CGRA-Based Baseband Processor for Massive MIMO Detection

Guiqiang Peng, Leibo Liu, *Senior Member, IEEE*, Sheng Zhou, *Member, IEEE*, Shouyi Yin, *Member, IEEE*, and Shaojun Wei, *Fellow, IEEE*

***Abstract*— Communication systems' development requires service customization in aspects, such as standards, multiple-input multiple-output (MIMO) scales, and algorithms. The existing hardware designs for massive MIMO detection have difficulty in achieving both high flexibility and scalability with high hardware efficiency. This article proposes a baseband processor based on a dynamic coarse-grained reconfigurable array (CGRA) for massive MIMO detection. To efficiently support various algorithm features and requirements, three optimization techniques are proposed to achieve high flexibility and scalability. First, an on-demand matrix–vector systolic array is proposed to enable flexible and scalable matrix and vector operations, reducing memory accesses by 82%. Second, distributed multi-interaction data storage is designed for flexible data access and reusability. Finally, a continuable adaptive context information format is proposed to support different bit widths, operations, and extensions of MIMO systems, reducing context information by 67%. These techniques achieve the improvements of 1.33×, 1.34×, and 1.29× in energy efficiency and 1.21×, 1.18×, and 1.18× in area efficiency, evaluated by removing one technique at a time from the proposed architecture. Fabricated in a 28-nm CMOS technology, the chip achieves high flexibility and scalability in supporting various detection algorithms; various MIMO scales, such as 4 × 4, 32 × 32, and 128 × 8; and baseband processing tasks, such as filtering and fast Fourier transformation. When benchmarked on various detection algorithms, the processor achieves 1.64–2.92-Gb/s/W energy efficiency and 0.25–0.43-Gb/s/MG area efficiency, which are 2.78–28.54× and 2.05–14.43× those of state-of-the-art programmable designs, respectively. To our knowledge, this is the first flexible and scalable CGRA-based baseband processor for massive MIMO detection.**

***Index Terms*— Coarse-grained reconfigurable architecture (CGRA), energy and area efficiencies, flexibility and scalability, massive multiple-input multiple-output (MIMO) detection, reconfigurable baseband processor.**

Manuscript received April 18, 2019; revised August 22, 2019; accepted November 2, 2019. Date of publication November 25, 2019; date of current version January 28, 2020. This article was approved by Associate Editor Edith Beigne. This work was supported in part by the National Natural Science Foundation of China under Grant 61834002, in part by the National Key R&D Program of China under Grant 2018YFB2202101, and in part by the National Science and Technology Major Project of the Ministry of Science and Technology of China under Grant 2016ZX01012101. *(Corresponding author: Leibo Liu.)*

G. Peng, L. Liu, S. Yin, and S. Wei are with the Institute of Microelectronics, Tsinghua University, Beijing 100084, China (e-mail: liulb@tsinghua.edu.cn).

S. Zhou is with the Department of Electronic Engineering, Tsinghua University, Beijing 100084, China.

Color versions of one or more of the figures in this article are available online at http://ieeexplore.ieee.org.

Digital Object Identifier 10.1109/JSSC.2019.2952839

I. Introduction

MASSIVE multiple-input multiple-output (MIMO) is a key technology for future wireless communication systems, such as 5G [1]–[3]. Massive MIMO detection is one of the most critical baseband processing tasks. For the development of communication technologies, it is necessary to achieve personalized and customized services based on various standards, quality-of-service requirements, MIMO scales, and algorithms. The ability to support multiple standards and multiple protocols is one of the key considerations in hardware circuit design. Therefore, a massive MIMO detection processor needs to be flexible, to adapt to the varying environmental conditions of different scenarios and to different requirements, and scalable, for reliable and seamless integration with rapidly developing and evolving baseband processing algorithms [4]. Various research efforts have been devoted to optimizing hardware architectures for either massive MIMO or traditional MIMO detection. A number of detector implementations have been proposed based on dedicated application-specific integrated circuits (ASICs), including nonlinear detectors [5]–[10] and linear detectors [11]–[18]. These ASIC implementations generally target a single detection algorithm, and they can achieve optimal or near-optimal area and energy efficiencies for the target algorithm. However, these ASIC implementations have quite limited flexibility and scalability. Another class of detectors is those based on general architectures, such as graphics processing units (GPUs) [19], [20]. These detectors achieve high flexibility and throughput, but they suffer from significant energy and area consumption, resulting in limited energy and area efficiencies. A reconfigurable processor for massive MIMO detection is a promising solution due to its high flexibility, scalability, and energy/area efficiencies.

Many reconfigurable designs for massive MIMO detectors have been achieved based on field-programmable gate arrays (FPGAs), application-specific instruction set processors (ASIPs), and digital signal processors (DSPs) [4], [5], [15], [17], [18], [21]–[24]. These designs achieve high energy and area efficiencies compared with general architectures, but they typically prevail only in one or two domains of the large design space encompassing throughput, energy, area, and flexibility. Most FPGAs provide low-level, fine-grained parallelism with a high degree of flexibility. However, the flexible logic and interconnections of FPGAs result in long reconfiguration time

0018-9200 © 2019 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL. Downloaded on June 04,2026 at 03:59:59 UTC from IEEE Xplore. Restrictions apply.

and high costs; consequently, an FPGA consumes a large amount of power [25]–[27]. DSPs are easy to program and have relatively low power consumptions, and they belong to the class of instruction-driven computing architectures. In each calculation process, instructions must inevitably be loaded from the instruction memory and parsed before calculations can be performed. Therefore, DSPs are flexible but relatively inefficient [28]–[30]. Very long instruction words can reduce instruction parsing, but it is still necessary in such an architecture to repeat the loading and parsing process when the instruction functions are changed. A coarse-grained reconfigurable architecture (CGRA) [31]–[33], a computational model that is not driven by instructions, is based on a data flow graph (DFG) and is oriented toward heterogeneous spatial computing. For a given configuration, a fixed circuit is formed, enabling repeated execution and thus approaching the efficiency of an ASIC. Therefore, CGRAs offer high resource utilization and data reuse rates. Several CGRA-based detector designs have been proposed, such as the node perturbation (NP) method [34], QR decomposition [35], and the minimum mean-squared error (MMSE) approach [36]. However, only a limited range of MIMO detection algorithms can be supported on these processors, and the dynamic range of the processed data is relatively fixed. Thus, the flexibility and scalability of these designs need to be improved. In addition, these processors are designed only for traditional MIMO systems, whereas massive MIMO systems introduce additional challenges for the design of reconfigurable architectures.

This article proposes a baseband processor based on a dynamic CGRA for massive MIMO detection. In this processor, reconfigurable processing element arrays (PEAs) with 4×4 customized reconfigurable processing elements (PEs) for wireless communications, single reconfigurable interconnections, an optimized shared memory (SM) system, and a hierarchical computation flow are used for the data path, and a hierarchical context controller and a prefetched context memory (CM) system are designed for the context path. To further improve flexibility and scalability with high efficiency, this article proposes the following three optimization techniques.

1) An on-demand matrix–vector systolic array (OMSA) based on PEAs is proposed, which includes five types of systolic arrays that can be dynamically configured based on context information. The OMSA can achieve various types of operations and a large number of matrix and vector operations, thus reducing memory accesses by 82% and maximizing the reuse of data.
2) A distributed multi-interaction data storage (DMDS) architecture is designed, which offers storage at three levels: the system, PEA, and PE levels. In addition, a ring architecture of memories is designed. The DMDS architecture enables flexible data access patterns and data reusability in massive MIMO systems to accelerate processing.
3) A three-level continuable adaptive context information (CACI) format is presented to enable the flexible extension of context information. The CACI format flexibly supports different bit widths, operations, iterations, and various extensions for different MIMO systems. In addition, the context information needed in massive MIMO detection algorithms can be reduced by 67%.

A prototype 28-nm test chip has been designed, which demonstrates a performance level of 1.54 Gb/s at 800 MHz while consuming at 521–545 mW. The chip can be programmed for a variety of parameters; MIMO scales, such as 4×4 to 128×8; and applications, such as massive MIMO detection, filtering, and fast Fourier transformation (FFT); detection algorithms, such as Cholesky decomposition (CHD) [12], LU decomposition (LUD) [13], weighted Jacobi iteration (WeJi) [17], sphere decoding (SD) [7], and NP [34]. The implementation results demonstrate the flexibility and scalability of the architecture. In addition, the processor achieves high energy and area efficiencies compared with the state-of-the-art programmable architectures, which is important for 5G systems [4], [37]. The proposed chip is designed for use in massive MIMO detection at the base station (BS) and for other baseband processing tasks at both the BS and mobile terminals.

This article is organized as follows. Section II analyzes the flexibility and scalability considerations. Section III describes the system architecture of the proposed processor. Section IV details the three optimization techniques. Section V presents the chip implementation and extensive measurement results. Section VI briefly introduces the software tool chains. Conclusions are drawn in Section VII.

II. Flexibility and Scalability Considerations

In an $N_r \times N_t$ MIMO system with N_t transmitters on the user side and N_r antennas on the BS side, the uplink system can be modeled as follows [8]: $\mathbf{y} = \mathbf{Hs} + \mathbf{n}$, where $\mathbf{H} \in \mathbb{C}^{N_r \times N_t}$ is the channel matrix, $\mathbf{s} \in \mathbb{C}^{N_t \times 1}$ is the transmitted signal vector, $\mathbf{n} \in \mathbb{C}^{N_r \times 1}$ is an additive white Gaussian noise vector , and $\mathbf{y} \in \mathbb{C}^{N_r \times 1}$ is the received signal at the BS. In massive MIMO detection, the main objective is to estimate the transmitted signal $\mathbf{s}$. Massive MIMO detection algorithms can be classified into two categories: nonlinear and linear. Nonlinear algorithms, such as K-best [5], SD [6], [7], expectation-propagation detection (EPD) [8], message-passing detection (MPD) [9], [10], and NP [34], [38], achieve near-optimal performance. Linear detection algorithms, such as CHD and LUD [11]–[14], Neumann series approximation (NSA) [14], Gauss–Seidel (GS) algorithms [15], [16], WeJi [17], and parallelizable Chebyshev iteration (PCI) [18], offer reduced computational complexity with near-optimal detection accuracy. The NP algorithm starts by obtaining an initial MMSE result, followed by a parallel NP scheme with the purpose of generating a list of candidate vectors around the MMSE vector. In addition, sorted QR decomposition is performed. The final detection result is obtained by applying the maximum likelihood (ML) criterion to the candidate list, i.e., finding the vector with the smallest Euclidean distance [34], [38]. This section analyzes various types of algorithms and applications to summarize the characteristics of massive MIMO detection and the flexibility and scalability considerations for the design of a reconfigurable architecture.

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL. Downloaded on June 04,2026 at 03:59:59 UTC from IEEE Xplore. Restrictions apply.

A. Common Computational Patterns in Massive MIMO Detection Algorithms

Massive MIMO detection algorithms include various complex-valued matrix and vector operations, and the proportions of such operations may be large in certain algorithms. For example, matrix multiplication, inversion, and decomposition operations as well as matrix–vector multiplication operations are required in the CHD- and LUD-based MMSE algorithms [12], [13], for QR decomposition in the K-best [5] and SD [6], [7] algorithms, and in the NSA [14] and MPD [9], [10] algorithms. Different algorithms require various types of operations, such as basic addition, subtraction, multiplication, and shift operations as well as signed complex-valued addition, multiplicative accumulation, and so forth. Multiplicative accumulation accounts for a large proportion of all operations. In addition, there are different sizes of operations. For instance, the sizes of the matrices and vectors involved are different for different MIMO systems; specifically, the sizes of the channel matrix $\mathbf{H}$ and the received vector $\mathbf{y}$ are affected. For example, the CHD- and LUD-based MMSE detection algorithms presented in [12] and [13] require the computation and decomposition of the Gram matrix $\mathbf{G} = \mathbf{H}^H\mathbf{H}$. In addition, the decomposition results are used to achieve matrix inversion, and matrix–vector multiplication is required to calculate the matched-filter vector $\mathbf{y}^{\mathrm{MF}} = \mathbf{H}^H\mathbf{y}$. In these calculations, many types of operations are required, most of which are signed complex-valued multiplicative and vector operations. Hence, the architecture must support operations of different types and sizes, particularly matrix and vector operations.

B. Various Operation Bit Widths and Formats

Different MIMO systems have different detection accuracy requirements. Therefore, various bit widths are needed to support these performance requirements. For example, in the NSA method [14], the operations have the bit widths of 22, 15, 12, and 8 bits; in NP [34], the bit width of the operations is 16 bits. In addition, these algorithms require bitwise logic, such as bitwise AND, OR, and XOR operations and, therefore, also require a bit width of 1 bit. Wireless systems involve various combined vector–scalar and complex-real operations, such as those required in the GS [15], WeJi [17], and PCI [18] algorithms. These algorithms also require noncomputational operations, such as data alignment after multiplicative accumulation and during data transfers of various data widths and formats. For example, in the NP algorithm [34], complex-valued vector operations are required, such as in the calculation of the matched-filter vector, whereas the sorting and arithmetic square root operations in the QR decomposition and log-likelihood ratios require scalar operations. Hence, it is necessary to design reconfigurable computing units that can support operations of various bit widths and formats to fully exploit the computing power of the hardware resources. Additionally, efficient configuration mechanisms are expected to offload noncomputational operations, e.g., data alignment, from the processing units.

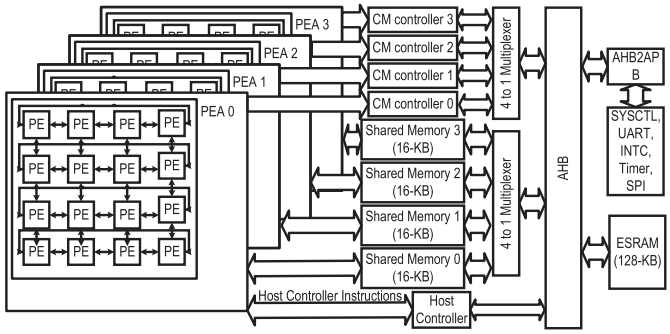


Fig. 1. Overall architecture.

C. Flexible and Efficient Data Access Patterns and Data Reusability

To support diverse applications and algorithms, various data formats are required, resulting in a variety of data access patterns. Massive MIMO detection algorithms involve vector, scalar, real-valued, and complex-valued data and operations; therefore, the architecture needs to efficiently support these different data formats and access patterns. To further exploit data-level parallelism, multi-subcarrier processing requires various data access patterns to allow diverse operations to be performed simultaneously on multiple subcarriers. Therefore, flexible memory access schemes are required. In addition, diverse data reuse patterns exist among massive MIMO detection algorithms. For example, in explicit algorithms, such as CHD- and LUD-based MMSE detection [12], [13], the Gram matrix computation results can be reused because of the channel hardening in massive MIMO systems. In implicit algorithms, such as PCI [18], the Gram matrix is not required, but the channel matrix $\mathbf{H}$ is reused many times. Hence, a flexible and efficient configurable on-chip memory system is required.

III. System Architecture

A. Overall Architecture

Fig. 1 shows the top-level architecture of the proposed processor. There are five main units, including four PEAs, embedded static random access memory (ESRAM), a host controller, SM for data storage, and CM controllers. In addition, this architecture includes common peripherals. The four 4×4 PEAs are the main computing units, which are the key components for implementing task acceleration. Each PEA can be dynamically partitioned into subarrays for computations with high algorithm-level parallelism. Each PE can be configured to execute operations of certain bit widths according to the CM in the PE. The design principle of the PEAs and PEs in the reconfigurable processor is to satisfy the requirements of the algorithm and circuit implementations to the greatest possible extent. According to the algorithm analysis, for a traditional MIMO system, four antennas are usually used, while for massive MIMO systems, 64-antenna, 128-antenna, and other larger antenna configurations may be used [1]–[3]. In addition, antenna configurations in powers of two are often discussed and used in related works [8]–[10], [12]. Accordingly, the dimensions of the PEAs need

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL. Downloaded on June 04,2026 at 03:59:59 UTC from IEEE Xplore. Restrictions apply.

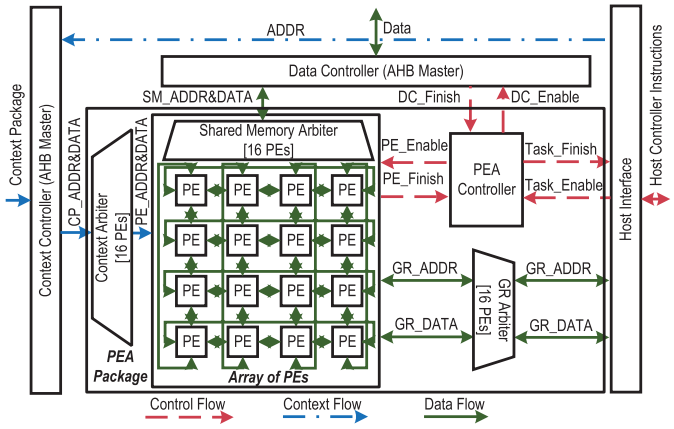


Fig. 2. Architecture of a reconfigurable PEA (CP: context package, DC: data controller, and ADDR: address).

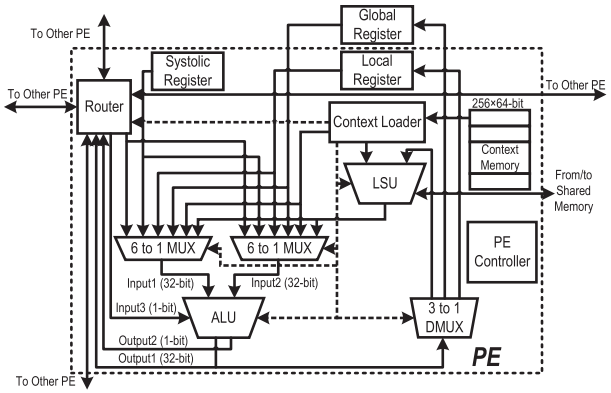


Fig. 3. Architecture of a customized reconfigurable PE.

to be compatible with both traditional and massive MIMO systems while avoiding the problems caused by redundant address coding (without using powers of two for the PEAs). If the size of the PEAs is large and the control logic is complex, the interconnections between the PEAs will also be complex, leading to memory access conflicts. To reduce these memory access conflicts, the SM needs to be larger and more complex, which will further increase the complexity of the control logic and may lead to a decrease in frequency. Based on an analysis of the tradeoff between the algorithm and circuit requirements, four 4 × 4 PEAs are suitable. The ESRAM is 128 kB in size, which includes the data and context information for all PEAs. The host controller, which is used to control the configuration and data for the PEAs and assign tasks to the PEAs for various functions by means of instructions, is implemented using ARM 7. The ARM processor requires a 4-bit enabling signal to enable the four PEAs, and 1 cycle is needed to send commands to the PEAs. The data and CM sizes are 64 and 128 kB, respectively. In the proposed architecture, there are two important paths: the data path and the context path.

B. Data Path

1) Reconfigurable PEAs: In the proposed processor, there are four reconfigurable PEAs, each of which includes a host interface, a context controller, a data controller, and the most important component, the PEA package (see Fig. 2). The host interface receives the instructions from the host controller to control the PEA package, and a small amount of necessary data from the host controller is written to the global register (GR) in the host interface. The context controller parses context information in the context package from the ESRAM and moves it to the CM. Then, the context information is sent to the PEA package to schedule the context execution sequence on the PEs. The data controller is responsible for data interaction between the PEA package and the SM. The PEA package consists of a PEA controller, a GR arbiter, a context arbiter, and an array of PEs. When the PEA controller receives the enable signal, all PEs are enabled. Then, the PEs execute the context information. After all PEs finish executing the context information, the PEA controller sends a signal to the host interface and waits for the next instruction from the host interface. The GR and context arbiters are used to arbitrate the GR data and the context package for each PE. The array includes 4 × 4 PEs, which work independently, thus providing a high level of parallelism to improve the throughput. In addition, each PE can be turned off to reduce redundant operations and power consumption. The power overhead for the unused PEs in each function is reduced by clock gating.

2) Customized Reconfigurable PEs for Wireless Communication: The architecture of a customized reconfigurable PE for wireless communication is shown in Fig. 3. In each PE, there are nine components, including a PE controller, a context loader, a router, an arithmetic logic unit (ALU), a load store unit (LSU), two systolic registers (SRs) for Input1 and Input2, 16 local registers (LRs), a 256 × 64-bit CM, and multiplexers (including a demultiplexer). The PE controller controls the processing and calculations performed by the PE. The context loader parses context information and defines the function of the whole PE. The router is used to transfer data between the PEs. The LSU is the memory access port of the PE. Depending on the context, this unit is defined to load or store data for or from the ALU from or in the SM. The main computing element, the ALU, and its inputs and outputs are controlled based on the context. The ALU is designed with 26 different functions, including logical functions, such as AND, OR, and XOR, as well as arithmetic functions, such as signed addition, subtraction, multiplication, and square root operations. Considering the complex-valued nature of massive MIMO systems, the ALU also includes complex-valued functions, such as signed complex addition, subtraction, and multiplication operations. Moreover, because of the large proportion of the total computations involving complex-valued matrix and vector operations, the ALU includes special functions, such as signed complex multiplication and accumulation and conjugate subtraction and accumulation. The functions of the ALU are represented by operation code (OpCode), which is summarized in Table I. To reduce the OpCode and context information, when the high 16 bits are zero, the arithmetic left/right shift and signed division operations are real-valued operations, whereas when the high 16 bits are not zero, these operations are complex-valued operations. Both coarse- and fine-grained computations are required in the ALU to accelerate both types of operations. The ALU includes

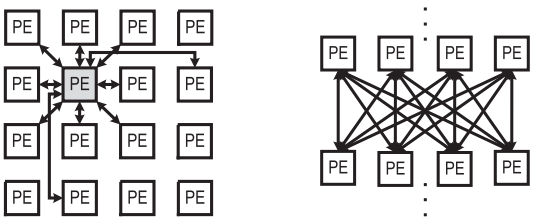
Authorized licensed use limited to: UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL. Downloaded on June 04,2026 at 03:59:59 UTC from IEEE Xplore. Restrictions apply.

TABLE I
FUNCTIONS OF THE ALU

No.	Function	OpCode	No.	Function	OpCode
0	NOP	00000	1	Signed addition	00001
2	Signed subtraction	00010	3	Signed multiplication	00011
4	AND	00100	5	OR	00101
6	XOR	00110	7	Selection	00111
8	Logic left shift	01000	9	Logical right shift	01001
10	Register cleared	01010	11	Square	01011
12	Extreme value	01100	13	Square root	01101
14	Data splicing (low 16 bits)	01110	15	Signed complex addition	01111
16	Signed complex subtraction	10000	17	Signed complex	10001
18	Signed complex multiplication and accumulation	10010	19	Signed complex subtraction and accumulation	10011
20	Signed complex conjugate multiplication	10100	21	Signed complex conjugate multiplication and accumulation	10101
22	Signed complex conjugate subtraction and accumulation	10110	23	Arithmetic left shift	10111
24	Arithmetic right shift	11000	25	Signed division	11001

two 32-bit data paths and one 1-bit data path, which can be combined to suit various bit width requirements. In addition, 32-bit data and 1-bit data can be cross-computed, as in the selection and extreme value functions (No. 7 and No. 12 in Table I). To support flexible data access and reusability (as analyzed in Section II-C), one solution is flexible input and output of the ALU. Thus, there are various alternatives for the 32- and 1-bit data paths. Note that the ALU performs only fixed-point operations and all ALU operations require two cycles because of the inclusion of division and square root operations, which are needed in some algorithms.

An 8×8 PEA architecture was proposed in [33]; however, the architecture is not sufficiently flexible for MIMO signal processing. Compared with the PEA architecture proposed in this article, the support provided for matrix and vector processing in [33] is insufficient. While the architecture proposed in [34] and [35] can be adapted to different MIMO scales, a given chip design will be able to support only a single MIMO scale. By contrast, the design in this article supports diverse MIMO scales. The PEA dimensions in [33]–[35] are also relatively large, which will lead to problems in the data interactions between the PEs and increase the cost of the interconnections; this will be further discussed in a quantitative manner in Section III-B3. Zhang *et al.* [34], [35] designed a PE for complex-valued operations for MIMO systems. However, the PE can realize only a few simple operations. Other functions, such as perturbation and division, are implemented by means of a dedicated coprocessor circuit accelerator. In the mapping process, there will be many data interactions between the PE and these dedicated circuits, which will greatly affect the computing efficiency of the processor. By contrast, the PE proposed in this article is specifically customized for wireless communications and offers high flexibility for achieving diverse types of operations. Compared with the design in [34] and [35], the proposed PE is more flexible and can greatly reduce processor latency. In the CGRA architectures presented in [32], [33], and [36], the proposed PEs support only real-valued processing. Considering the large numbers of complex-valued operations required, these general PEs are limited in performance. The customized PE architecture proposed in this article can quickly complete calculations, reducing the number of cycles required. Moreover, compared with [32], [33], and [36], the PE design presented in this article includes a variety of register architectures (including GR, SR, and LR modules), six PE input sources (as shown in Fig. 3), and two units (ALU and LSU). This design can thus provide greater flexibility and scalability and is therefore conducive to the implementation of subsequent optimization techniques (see Section IV). For example, the SR design is conducive to the implementation of the OMSA technique, and the LSU design is conducive to the implementation of the PE pipeline.



(a) Two-hop connection (b) Line connection

Fig. 4. Comparison of different interconnections. (a) Two-hop connection. (b) Line connection.

3) Single Reconfigurable Interconnections: Reconfigurable interconnections between PEs are achieved by means of a router with a nearby mechanism. Through the router, a PE can access the calculation result from a neighboring PE in the previous cycle. Each PE can be connected to four adjacent PEs (i.e., up, down, left, and right) and itself, as shown in the array of PEs in Fig. 2, and this entire reconfigurable interconnection is called a single connection. Note that the PEs in the first and fourth rows are interconnected and the PEs in the first and fourth columns are interconnected, as shown in Fig. 2. The interconnections between the PEs can be dynamically changed based on the context information to realize multiple types of data flows inside the PEA. Therefore, these reconfigurable interconnections can flexibly support multiple types of data transmission. This flexibility also ensures the implementation of different algorithms, different antenna sizes, and the extension of algorithms in the massive MIMO detection.

Two alternative reconfigurable interconnection strategies have been proposed in the state-of-the-art work [32], [33]. In the first type of connection, called a two-hop connection, each PE can be connected to any PE within two steps [32], as shown in Fig. 4(a). In the second, called a line connection, each PE can be connected to any PE in the adjacent lines above and below, as shown in Fig. 4(b) [33]; in this strategy, the last line is interconnected with the first line. To compare these reconfigurable interconnection strategies, a PEA module has been implemented. Fig. 5 shows the ratios of the interconnection area to the PEA area with the different types of connections. Fig. 5 shows that the single connection strategy results in lower area consumption compared with the two-hop and line connection strategies [32], [33]. Considering the importance of architectural exploration, the advantage of the single connection approach is even more prominent. Note that the measured results show that the throughputs of different massive MIMO detection algorithms are not different with the

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL. Downloaded on June 04,2026 at 03:59:59 UTC from IEEE Xplore. Restrictions apply.

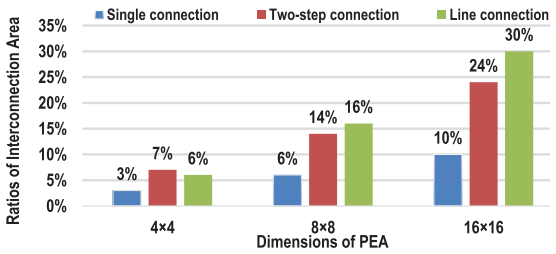


Fig. 5. Ratios of interconnection area to PEA area with different PEA dimensions.

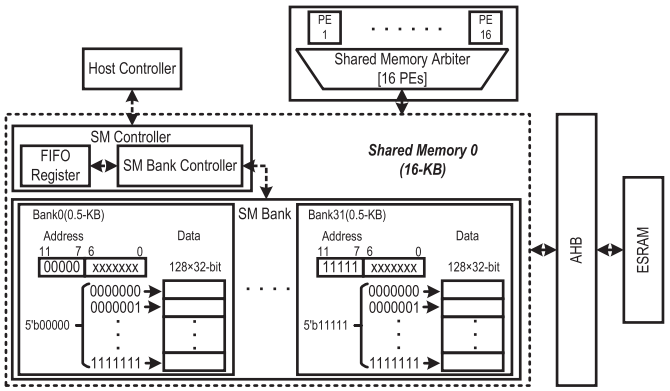


Fig. 6. Architecture of an SM unit.

single connection strategy than they are with the two-hop and line connection strategies because single connections satisfy the requirements for the PE interconnects for massive MIMO detection algorithms, according to the algorithm analysis. The main purpose of the router is to share the output results of the PEs. Matrix and vector operations account for the majority of the algorithm computations. Since the ALU includes the functions of complex multiplication and accumulation, the calculation results from the ALU in the same PE are mostly used when performing matrix and vector multiplication, and the results are only seldom transmitted to other PEs for calculations. Therefore, the single connection strategy has a little impact on the system throughput. For the example of an 8×8 complex matrix multiplication, all three interconnection strategies need the same number of cycles, i.e., 36.

4) SM System: An SM is a multibank memory, and one SM unit consists of two modules: an SM controller and an SM bank, as shown in Fig. 6. The SM controller is the data-handling control module, and the SM bank is the data storage module. The configuration of data transfer is controlled by the SM bank controller in accordance with the internal first-input first-output (FIFO) register of the SM controller. The SM controller module automatically completes the data transfer in accordance with the specified configuration and notifies the host controller and PEA. When the PEA and SM controller attempt simultaneous access, the SM controller has the higher priority, and the PEA request is held. Each SM has a capacity of 16 kB, and each bank has a capacity of 512 bytes with 128×32 bits. Each bank has a 12-bit address, where the first 5 bits are a tag. The priorities of multiple PEs accessing the same bank at the same time are determined by the arbiter. The arbiter supports broadcast operations. Therefore, if multiple PEs initiate data read requests for the same address, the arbiter

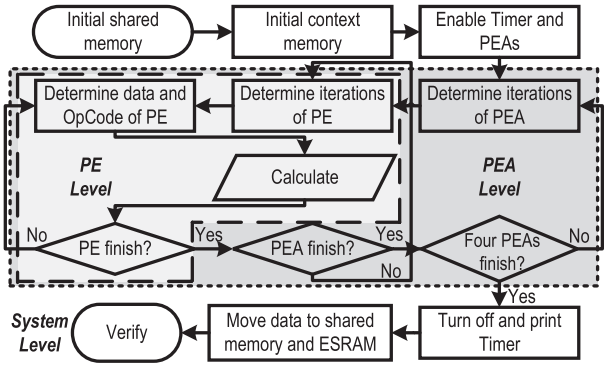


Fig. 7. Hierarchical computation flow.

needs only one cycle to satisfy all requests. For example, for the multiplication of numbers by vectors, a broadcast operation can be used to prepare the input data for multiple PEs.

5) Hierarchical Computation Flow: In this processor, the computation flow (see Fig. 7) is divided into three levels: the system, PEA, and PE levels. First, the host controller controls the ESRAM to transfer data to one of the SMs. After all data are ready, the start address and length of the context package are transferred to one CM. After all data and context information for the PEAs are ready, the timer is initialized, and the timer and PEAs are enabled. Each PEA reads the PEA context information to determine its overall number of iterations. Each PE in the PEA reads its corresponding context information and determines its number of iterations . Then, each PE reads the contents of the context line and performs its computation until all 16 PEs have finished computing, that is, the PEA completes one round of computation. When all four PEAs finish their iterative calculations, the timer is turned off, and the number of clock cycles is printed. The required data are moved to the SM from the PEAs, and the host controller moves the SM data into the ESRAM. Then, the correctness is verified, and the results are printed. Finally, the PEAs are turned off to reduce power consumption and prepare for the subsequent operations.

C. Context Path

1) Hierarchical Context Controller: The computational kernels are organized as the context information and distributed at three levels, from top to bottom. First, at the system level, the context word is parsed; this is the context information received by the interface. Second, at the PEA level, the top-level context information for each PEA is read. This information mainly describes the overall information for the PEA, such as the number of times that the context package for each PEA must be executed. Third, at the PE level, each of the 16 PEs in each PEA reads its own top-level context information, which includes the overall information for each PE, such as the total number of lines corresponding to each PE in the context package and the number of times that the context package for each PE must be executed. Then, the context information for each PE is read, which includes the basic information for the PE, such as the inputs, outputs, and operations. The transfer of the context information for a PE is complete once the total

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL. Downloaded on June 04,2026 at 03:59:59 UTC from IEEE Xplore. Restrictions apply.

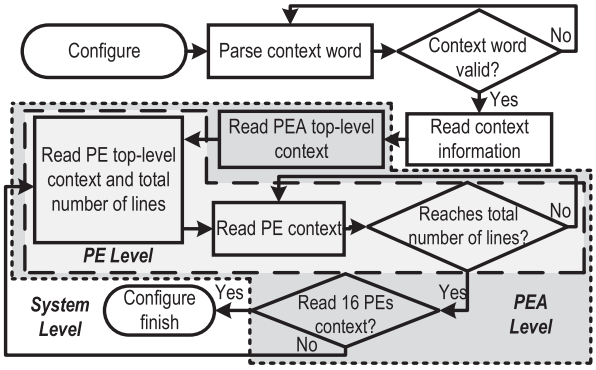


Fig. 8. Hierarchical context controller flow.

number of lines of context information associated with that PE is reached. For example, suppose that the context information for PE1 is contained in lines 15–32 in the context package (the line numbers are determined from the PE top-level context information); thus, once the context information on line 32 has been read, the transfer of the context information for PE1 is finished. Once the context information for all 16 PEs has been transferred, the transfer of all context information for the PEA is complete. The hierarchical context controller flow is shown in Fig. 8. The reconfigurable processor is driven by the context data stream pipeline (CDSP). The context flows, data flows, and calculations are performed in parallel in a pipelined manner. There are five segments in the processing pipeline: fetch, decode, load data, execute, and write back. First, context information is fetched from the CM through the context loader. Then, a decoding operation is performed to analyze the context information. In accordance with the context information, the ALU's input data are loaded, and then, the calculation is performed. When the calculation is finished, the result is output to the PE, and the processing is complete. Each segment of the pipeline requires two clock cycles. Except for the ALU segment, there are two operations in each segment, such as fetching an address and the corresponding data.

Table II shows an evaluation of the CDSP technique. The data in Table II present a comparison between the architecture with all proposed techniques and without the CDSP. The system throughput is significantly improved with pipeline processing. Specifically, on the LUD, CHD, WeJi, and NP benchmarks, the architecture with the CDSP achieves 3.02× higher throughput on average. Although the power consumption is also increased on these four benchmarks, the architecture with the CDSP achieves 1.25× higher energy efficiency on average. Moreover, because of the significant increase in throughput, the architecture with the CDSP also achieves 3.02× higher area efficiency on average.

2) Prefetched CM System: A CM unit is distributed within each PE, and the size of each CM unit is 256 × 64 bits. In the process of reconfigurable array configuration, a fully dynamic, reconfigurable, 3-D context information technique is used. Here, the term 3-D refers to the extension of the PEAs, which occupy two dimensions in space, along the time axis. This technique enables the context information for a PE unit to be

TABLE II
EVALUATION OF THE CDSP TECHNIQUE

Benchmark		LUD	CHD	WeJi	NP
MIMO Size		4×4	128×8	128×8	4×4
Throughput [Gb/s]	with CDSP	1.2	0.88	1.54	0.97
	w/o CDSP	0.4	0.29	0.51	0.32
Power [W]	with CDSP	0.545	0.537	0.528	0.521
	w/o CDSP	0.232	0.221	0.217	0.21
Area (Logic Gates) [MGE]	with CDSP	3.55	3.55	3.55	3.55
	w/o CDSP	3.55	3.55	3.55	3.55
Energy Efficiency [Gb/s/W]	with CDSP	2.202	1.639	2.917	1.862
	w/o CDSP	1.724	1.312	2.350	1.524
Area Efficiency [Gb/s/MGE]	with CDSP	0.338	0.248	0.434	0.273
	w/o CDSP	0.113	0.082	0.144	0.090

replaced periodically without any explicit configuration delay during operation. In general, several sets of nonsequential context information are used to complete tasks in a fixed order of execution. The ranking of the context information within a set can be parsed and used to accurately predict the next context information so that it can be prefetched. In addition, because the PEA uses a pipeline mechanism for configuration and calculation, the context information for the next cycle will be prefetched during the process of executing the calculation of the PE, enabling dynamic and real-time PE configuration. In this way, pipeline processing of the PE context information is realized. No similar prefetching technique is used in [34]–[36] to optimize the process of configuring the CGRA architecture. The experimental mapping results for the LUD, CHD, WeJi, and NP algorithms show that this prefetching technique reduces the configuration delay by approximately 36% on average compared with the results without prefetching.

IV. OPTIMIZATION TECHNIQUES

In this section, three techniques are proposed for optimizing the reconfigurable architecture for massive MIMO detection. These optimization techniques are based on the features of the algorithms as well as the flexibility and scalability requirements analyzed in Section II When achieving high flexibility and scalability, the energy and area efficiencies are also improved. Note that the optimization techniques proposed in this section are evaluated by comparing the proposed architecture (including all techniques) with the corresponding architecture without the technique that is being evaluated.

A. OMSA

1) PEA-Based OMSA Architecture: As described in Section II, a large number of matrix and vector operations are needed in massive MIMO detection algorithms. In these calculations, different PEs need to access different addresses in the same bank during the same clock cycle. Hence, a large number of memory access conflicts may arise. To optimize the architecture for a large number of matrix and vector operations, this article proposes an OMSA architecture based on PEAs. Note that the purpose of the SM arbiter is to solve the problem of different PEs accessing the same address in the same bank during the same clock cycle. The OMSA technique is designed to resolve the conflicts among different PEs accessing different addresses in the same bank during the same clock cycle. There are five types of systolic arrays that

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL. Downloaded on June 04,2026 at 03:59:59 UTC from IEEE Xplore. Restrictions apply.

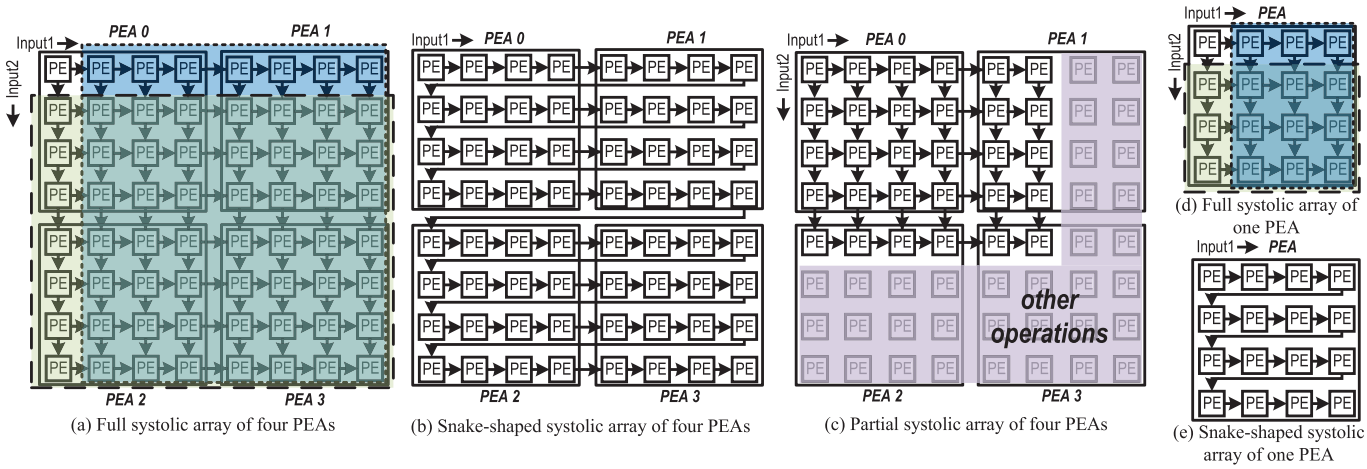


Fig. 9. Architectures of different types of systolic arrays based on PEAs. (a) Full systolic array of four PEAs. (b) Snake-shaped systolic array of four PEAs. (c) Partial systolic array of four PEAs. (d) Full systolic array of one PEA. (e) Snake-shaped systolic array of one PEA.

can be selected based on the algorithm and the calculation scale.

The first type is a full systolic array of four PEAs, as shown in Fig. 9(a). In this configuration, all four PEAs are connected to form a large 8 × 8 systolic array. The Input1 data for each PE on the left side of the array are read from the SM. At the same time, the Input1 data are stored in the SR of each PE on the left side. In the next cycle, the second PE reads the Input1 data for the SR for calculation, and the Input1 data are saved to the SR of the second PE. By analogy, in the 8 × 8 systolic array, for the Input1 data of the PEAs, SM accesses are necessary only for the left eight PEs. Then, the Input1 data are transmitted horizontally, and the PEs in the 8 × 7 array on the right do not need to obtain the Input1 data from the SM. Similarly, for the Input2 data of the 8 × 8 systolic array, only the eight PEs in the first row need to obtain the data from the SM. Then, the Input2 data are transmitted vertically, and the PEs in the lower 7 × 8 array do not need to obtain the Input2 data from the SM. The second type of array is a snake-shaped systolic array of four PEAs, as shown in Fig. 9(b). In this configuration, the four PEAs are connected to form a 1-D computing unit array, and only the first PE needs to access the SM for Input1 data. The Input1 data for the other PEs are obtained from the previous PE. The third type of array is a partial systolic array of four PEAs, as shown in Fig. 9(c). This architecture is obtained by cropping the first full systolic array of four PEAs. In this way, the four PEAs can be configured into systolic arrays of different sizes in accordance with the context information, and the remaining PEs can perform other operations. The fourth and fifth array types are based on a full systolic array and a snake-shaped systolic array, respectively, formed from a single PEA, as shown in Fig. 9(d) and (e). The calculation processes for these two configurations are similar to those for the four-PEA systolic arrays (the first and second types of systolic arrays).

2) Evaluation of the OMSA: To assess the advantages of the different types of systolic arrays, an 8 × 8 complex matrix multiplication was mapped onto the architecture. The evaluation was performed by comparing the experimental data for the architectures with and without the systolic array of interest. With the full systolic array, the number of SM accesses on the input data side can be reduced from 1024 to 120 (by approximately 88%). In addition, memory access conflicts can be significantly reduced. With the snake-shaped systolic array, the number of SM accesses is reduced by 56% on the input data side (from 1024 to 448), and there is no memory access conflict for input data. This array type can meet the needs of large-scale complex-valued vector calculations in massive MIMO detection algorithms. A partial systolic array of four PEAs can be used to perform various irregular complex-valued matrix and vector operations. For a single PEA, the full and snake-shaped systolic arrays reduce the number of SM accesses on the input data side by 78% (from 1024 to 224) and by 50% (from 1024 to 512), respectively. At the same time, access conflicts are also greatly reduced.

Systolic arrays that are specifically designed for a certain target MIMO scale and algorithm are presented in [8], [12], [14], [17], and [18]. These architectures are fixed and inflexible, and they cannot be adaptively adjusted. By contrast, the OMSA architecture is highly flexible, and it can be reconfigured to form different types of systolic arrays that are suitable for different MIMO scales and algorithms. Moreover, this architecture is optimized for matrix and vector processing. The CGRA architectures presented in [32]–[36] are not designed for MIMO systems and do not include systolic arrays; consequently, they suffer from many memory access and memory conflicts during matrix and vector operations. Table III shows the evaluation results for the proposed architecture (with all techniques) and the corresponding architecture without the OMSA technique. The analyses are based on the LUD, CHD, WeJi, and NP benchmarks for 4 × 4 and 128 × 8 MIMO systems. According to Table III, by virtue of the OMSA technique, the numbers of memory accesses in these four algorithms are reduced by 77%, 88%, 88%, and 76%, respectively (82% on average). The numbers of memory access conflicts in the LUD and NP algorithms for the 4 × 4 MIMO system decrease from 68 and 72, respectively, to 0, and the numbers of memory access conflicts in the CHD and WeJi algorithms for the 128 × 8 MIMO system decrease by 89% and 92%, respectively. Therefore, the PEA

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL. Downloaded on June 04,2026 at 03:59:59 UTC from IEEE Xplore. Restrictions apply.

TABLE III
EVALUATION OF THE OMSA TECHNIQUE

Benchmark		LUD	CHD	WeJi	NP
MIMO Size		4×4	128×8	128×8	4×4
Data Memory Accesses [times]	with OMSA	36	1936	1868	42
	w/o OMSA	158	16450	15368	174
Access Conflicts [times]	with OMSA	0	156	106	0
	w/o OMSA	68	1364	1250	72
Throughput [Gb/s]	with OMSA	1.2	0.88	1.54	0.97
	w/o OMSA	1	0.72	1.3	0.8
Power [W]	with OMSA	0.545	0.537	0.528	0.521
	w/o OMSA	0.61	0.597	0.577	0.573
Area (Logic Gates) [MGE]	with OMSA	3.55	3.55	3.55	3.55
	w/o OMSA	3.55	3.55	3.55	3.55
Energy Efficiency [Gb/s/W]	with OMSA	2.202	1.639	2.917	1.862
	w/o OMSA	1.639	1.206	2.253	1.396
Area Efficiency [Gb/s/MGE]	with OMSA	0.338	0.248	0.434	0.273
	w/o OMSA	0.282	0.203	0.366	0.225

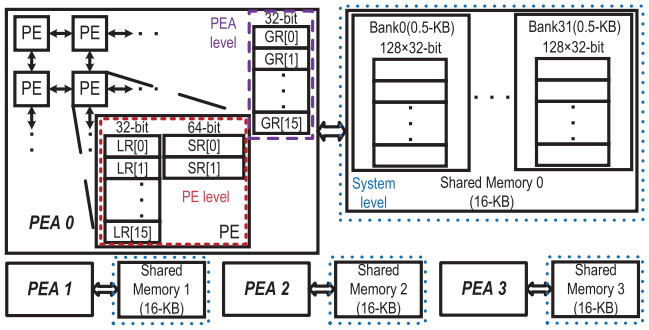


Fig. 10. DMDS architecture.

computing latency is reduced, and the processor achieves 1.21× higher throughput on average. Due to the reduction in memory accesses, the power consumption of the processor is also reduced by 10% on average. Hence, the system achieves 1.33× and 1.21× higher energy and area efficiencies on average, respectively.

B. DMDS

1) DMDS Architecture: Based on the algorithm analysis, the DMDS architecture presented in this article is separated into three levels, namely, the system, PEA, and PE levels, as shown in Fig. 10. The system-level data storage mainly corresponds to the on-chip SMs. Each SM can store input-output data for two PEAs. PEA-level storage is provided by the GRs. Each PEA has 16 GRs, each of which is 32 bits in size; thus, the GRs of all four PEAs have a total size of 2 kb. Each GR is mainly used to store data and parameters that the related PEA needs to write from the host controller. These data and parameters generally need to be calculated in multiple PEs. PE-level storage is provided by the LRs and SRs. Each PE has 16 internal LRs, each of which has a size of 32 bits; thus, the total size of all LRs is 4 kB. The LRs are mainly used to store intermediate data inside each PE. In each PE, there are two 64-bit SRs at Input1 and Input2 to support OMSA processing (discussed in Section IV-A). Each PEA can access two SMs. The PEAs and SMs are designed in a ring architecture, as shown in Fig. 11. Each SM contains 32 banks, each of which is accessed independently. Each bank corresponds to a separate SRAM in the physical implementation, meaning that each SM can satisfy up to 32 access requests per clock cycle.

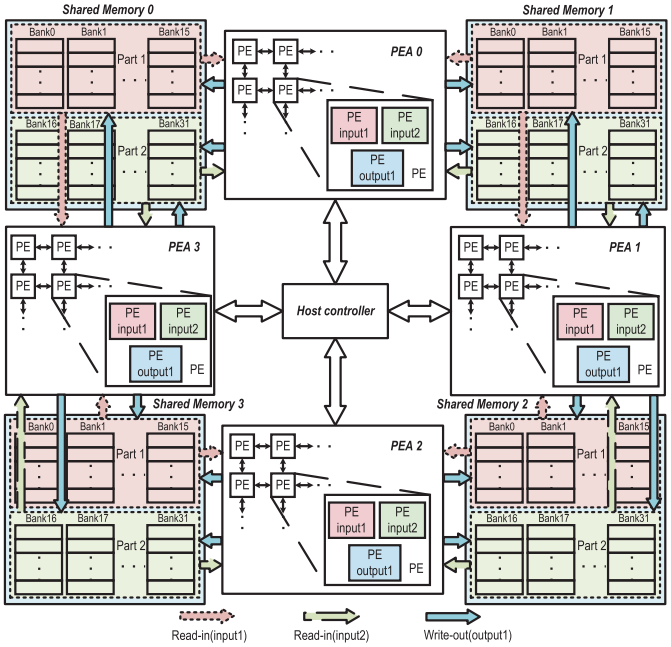


Fig. 11. Ring architecture of the SMs.

2) Evaluation of DMDS: By means of the DMDS architecture, various read and write granularities, bandwidths, and memory access modes can be realized to adapt to different data access mechanisms. At the same time, the different levels can support different types of data reusability. In addition, because of the ring architecture of the SMs, there is no need to switch out the context information for an entire task at once, and there is no additional data transfer time. For a 4 × 4 PEA, Input1 and Input2 of all PEs may require up to 16 input data to be read from the SM simultaneously. There are 32 SM banks to store data, and the SM addresses accessed by Input1 and Input2 do not overlap. For the output of a PE, there are 64 banks available for storing data and writing out to the left/right SMs. Therefore, this design can greatly reduce SM access conflicts. In addition, compared with the case in which each PEA separately accesses one SM, the ring architecture of SMs provides stronger data mobility. The CGRA architecture designed in [34]–[36] contains independent storage systems. By contrast, the DMDS architecture presented in this article allows the storage mode of the processor to be optimized at multiple levels to realize intralevel and interlevel multi-interactivity among the PE, PEA, and system levels. The DMDS technique reduces memory access delays and power consumption while achieving high flexibility and scalability. Table IV presents the evaluation results for the proposed architecture (with all techniques) and the corresponding architecture without the DMDS technique. The results show that DMDS enables 1.09× higher throughput, on average, on the different benchmarks. Due to the flexible data storage organization and PE design, the proposed processor can support the operations of various bit widths in massive MIMO detection algorithms. According to Table IV, the proposed processor can support operations with the bit widths of 1, 2, 8, 12, and 16 bits. In addition, a comparison of the architectures with and without DMDS reveals that the DMDS technique reduces the power

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL. Downloaded on June 04,2026 at 03:59:59 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
EVALUATION OF THE DMDS TECHNIQUE

Benchmark		LUD	CHD	WeJi	NP
MIMO Size		4×4	128×8	128×8	4×4
Bit Width		1/2/16	1/8/12/15/22	1/12/14/18	1/16
Throughput [Gb/s]	with DMDS	1.2	0.88	1.54	0.97
	w/o DMDS	1.14	0.79	1.38	0.9
Power [W]	with DMDS	0.545	0.537	0.528	0.521
	w/o DMDS	0.67	0.662	0.647	0.639
Area (Logic Gates) [MGE]	with DMDS	3.55	3.55	3.55	3.55
	w/o DMDS	3.83	3.83	3.83	3.83
Energy Efficiency [Gb/s/W]	with DMDS	2.202	1.639	2.917	1.862
	w/o DMDS	1.701	1.193	2.133	1.408
Area Efficiency [Gb/s/MGE]	with DMDS	0.338	0.248	0.434	0.273
	w/o DMDS	0.298	0.206	0.360	0.235

and area consumptions of the processor by 19% and 7% on average, respectively. Accordingly, the proposed processor design achieves 1.34× and 1.18× higher energy and area efficiencies on average, respectively. The main reason for the area reduction achieved with DMDS is the hierarchical design of the storage architecture, which can reduce the overhead in terms of the number of redundant storage units. With DMDS, the calculation results from a PE need not be written back to the SM, which will lead to many conflicts. To avoid conflicts and maintain a high throughput and data accuracy, the size of the SM needs to be increased. This will also increase the area to some extent. Meanwhile, the ring architecture of SMs in DMDS incurs a small increase in interconnections. Overall, however, the DMDS technique results in reduced area consumption.

C. CACI

1) CACI Format: A task for a PEA is defined by a context package, which contains one line of PEA top-level context information (64 bits) and the individual context information for the 16 PEs. The context information for each PE contains one line of PE top-level context information (64 bits) and several lines of PE context information (64 bits). When the context information cannot be fully expressed in 64 bits, an extended context information line is adopted. Some less commonly used semantics are placed in the extended context information, such as 32-bit immediate values or iteration intervals greater than 8. In this way, it is ensured that all types of semantics can be fully expressed, and the context information will still be 64 bits in common cases. To compress the context information and achieve certain irregular loops, iteration numbers are provided in different levels of the context information such that the lines of context information can be repeatedly executed. Fig. 12 shows the three-level context information of the reconfigurable processor: PEA top-level, PE top-level, and PE context information. First, the iteration number for the PEA is expressed in 32 bits to specify the iterative calculations in the kernel; this number indicates the number of times that this context package must be executed. Second, the PE top-level context information is 26 bits in size, including the count of configuration lines, the iteration number, the starting line number, and the fixed-point scaling information. The count of configuration lines represents the total number of lines in the entire context package corresponding to this PE. To address the influence of initialization on the PE pipeline,

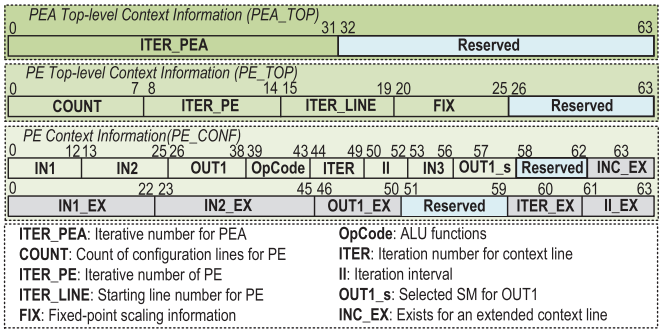


Fig. 12. CACI format for a PEA.

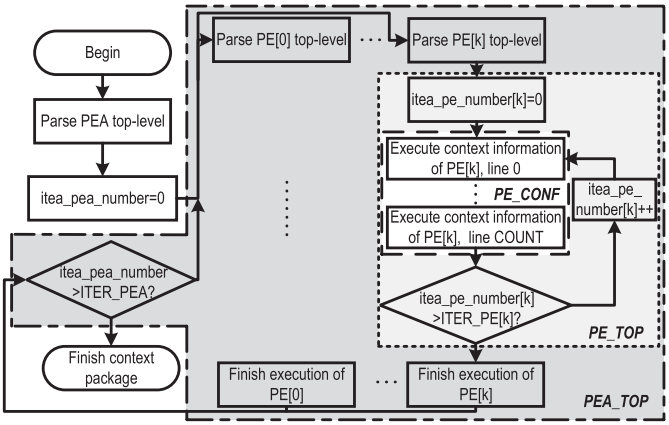


Fig. 13. Example of the three iterative loops for context information.

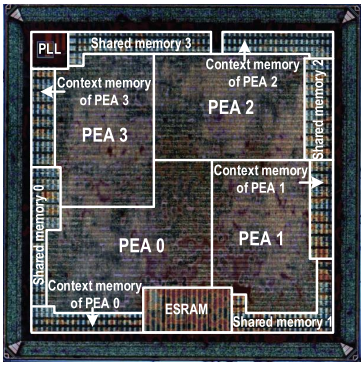
the starting line number for the PE is specified in the PE top-level information. The fixed-point scaling information specifies the shift information for complex-valued multiplication in the PE. In this way, all context information for complex-valued multiplication and shift operations in the PE can be compressed from two lines into one line. The scaling information is used to meet different numerical precision requirements. In addition, two PEs can be combined into one PE to further address a case of insufficient bit width. Third, the 59-bit PE context information controls the processing performed in the PE, including input, output, operation, iteration, and extension information. In accordance with the CACI format, the context information is executed in three iterative loops, as shown in Fig. 13. The outermost loop corresponds to the number of times that the entire context package for the PEA needs to be executed. The middle loop corresponds to the number of times that the context information for a given PE in the PEA needs to be executed. The innermost loop corresponds to the number of times that one context line for a PE needs to be executed. At all levels, the term iteration refers to performing the same task multiple times.

2) Evaluation of CACI: Zhang *et al.* [34], [35] and Chen *et al.* [36] proposed the CGRA architectures for massive MIMO detection. However, the context information format used in these architectures is not optimized. The CACI format designed in this article offers efficiently optimized organization and flexible support for the continuous extension of context information. Therefore, this CACI format supports the flexibility and scalability of the CGRA architecture at the context information level and can greatly reduce the redundancy of

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL. Downloaded on June 04,2026 at 03:59:59 UTC from IEEE Xplore. Restrictions apply.

TABLE V
EVALUATION OF THE CACI TECHNIQUE

Benchmark		LUD	CHD	WeJi	NP
MIMO Size		4×4	128×8	128×8	4×4
Context Information [KB]	with CACI	9.42	31.28	28.17	11.36
	w/o CACI	32.25	80.57	76.68	42.28
Throughput [Gb/s]	with CACI	1.2	0.88	1.54	0.97
	w/o CACI	1.1	0.81	1.33	0.83
Power [W]	with CACI	0.545	0.537	0.528	0.521
	w/o CACI	0.659	0.62	0.591	0.583
Area (Logic Gates) [MGE]	with CACI	3.55	3.55	3.55	3.55
	w/o CACI	3.72	3.72	3.72	3.72
Energy Efficiency [Gb/s/W]	with CACI	2.202	1.639	2.917	1.862
	w/o CACI	1.669	1.306	2.250	1.424
Area Efficiency [Gb/s/MGE]	with CACI	0.338	0.248	0.434	0.273
	w/o CACI	0.296	0.218	0.358	0.223



Technology	TSMC 28nm
Core Size [mm^2]	2.19×2.19
Logic Gates [MGE]	3.55
SRAM [KB]	320
Voltage [V]	0.9
Frequency [MHz]	800
Detection Supported	LU/CHD/WeJi/ SD/NP etc.
Power [mW]	521-545
Bit Width	1/2/8/12/14/15/ 16/18/22/32
Throughput [Gb/s]	0.88-1.54
Other Applications	NCO/FFT/FIR/ HBF/IDFT etc.

Fig. 14. Die micrograph and specifications.

context information and the context delay that is incurred when obtaining the address of the context information and retrieving the corresponding data. Table V shows the evaluation results for the proposed architecture (with all techniques) and the corresponding architecture without the CACI technique. With the use of the CACI format, the sizes of the context information for the LUD, CHD, WeJi, and NP benchmarks are decreased by 71%, 61%, 63%, and 73%, respectively (67% on average). Due to the reduction in configuration delay and the introduction of the iterative configuration, the system achieves a 1.13× increase in throughput on average, and the power and area consumption are decreased by 13% and 5% on average, respectively. Thus, the processor achieves 1.29× and 1.18× higher energy and area efficiencies on average, respectively.

V. CHIP IMPLEMENTATION AND COMPARISON

The proposed architecture was implemented on a 2.19 × 2.19 mm^2 silicon chip using the TSMC 28-nm CMOS technology. Fig. 14 shows the die micrograph and chip specifications. This section presents an evaluation of the proposed processor. Section V-A analyzes the processor's flexibility and scalability in several respects, including support for diverse algorithms, different MIMO scales, and other baseband processing tasks. Section V-B analyzes the energy and area efficiencies and compares them with those of state-of-the-art designs. Section V-C analyzes the power and area breakdowns of the processor.

A. Analysis of Flexibility and Scalability

1) Flexibility and Scalability in Terms of Algorithms Supported: The proposed chip can be flexibly configured to support linear and nonlinear detection algorithms. For comparison, several typical detection algorithms are considered in Tables VI and VII, such as CHD [12], LUD [13], WeJi [17], SD [7], and NP [34]. These algorithms were selected to test the processor's support for different algorithms, parameters, and MIMO scales (4 × 4 and 128 × 8). Hence, the implementation results illustrate the flexibility and scalability of the architecture. Note that massive MIMO detectors have also been implemented based on ASICs [8]–[13]. Table VI lists the results for two of the most representative massive MIMO detectors with ASIC implementations. Compared with the ASIC-based linear [12] and nonlinear [8] detectors, the energy and area efficiencies of the proposed architecture are 10%–12% and 12%–52% as high, respectively, which are reasonable results according to [34], [39], and [40]. It is possible for CGRA implementations to achieve efficiencies closer to those of ASICs, but this comes at the expense of flexibility. Applications of CGRA designs in other fields confirm this claim [33], [41]. ASICs have certain advantages in terms of energy and area efficiencies due to their highly customized architectures and memory systems. However, such an ASIC implementation must be designed for only a single target algorithm and MIMO scale. These architectures cannot be programmed in a flexible and scalable manner. With the continuing development of communication systems, the flexibility requirements are increasing, and the use of programmable hardware has become a common trend [4]. Therefore, the reconfigurable processor proposed in this article is advantageous for the current state of the art. Typically, if a detector designed for a large-scale MIMO system is used for a relatively small-scale MIMO detection, the resource overhead will be high, and the hardware utilization will be low. By contrast, a CGRA operates by configuring PEs for specific functions and releasing unused PEs to be configured for other functions, thus permitting high hardware utilization. In addition, it is difficult to define the maximum possible scale of an MIMO system. As MIMO systems scale up, ASIC implementations will encounter scalability problems. In addition, a change in the MIMO scale may also lead to a change in algorithm. In a CGRA, the final problem of how to execute the calculations for a large-scale MIMO system, such as a 256 × 16 and 16 × 16 system, can be solved by adjusting the PE scheduling mode, that is, the flexible allocation of limited resources can be successfully achieved to complete the calculations for MIMO detection at different scales.

2) Flexibility and Scalability in Terms of the MIMO Scale: With the development of various algorithms, diverse algorithms and scales of data need to be processed. As the scale of the MIMO system increases, the proposed chip can still be used to execute detection algorithms. For example, Table VII shows that the proposed architecture can be used to efficiently implement MIMO detection algorithms of different types and scales. The MIMO sizes supported range from 4 × 4 to 32 × 32. Hence, these experimental results show that the proposed processor supports MIMO scaling. For the same chip, it is normal for the system throughput to decrease as the MIMO scale increases because of the dramatic increase in complexity.

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL. Downloaded on June 04,2026 at 03:59:59 UTC from IEEE Xplore. Restrictions apply.

TABLE VI
COMPARISON WITH STATE-OF-THE-ART DESIGNS

	Linear detectors								Nonlinear detectors				
	[17]	[15]	[16]	[24]	**This Paper**	[12]	[23]	**This Paper**	[8]	[7]	**This Paper**	[34]	**This Paper**
Platform	**FPGA**				**CGRA**	**ASIC**	**FPGA**	**CGRA**	**ASIC**	**FPGA**	**CGRA**	**CGRA**	**CGRA**
Technology [nm]	28	28	28	28	**28**	28	28	**28**	28	28	**28**	65	**28**
Algorithm	WeJi	GS	C2PO	OCD	**WeJi**	CHD	CHD	**CHD**	EPD	SD	**SD**	NP	**NP**
SNR Loss[a] [dB]	0.49	1.98	1.67	1.73	**0.49**	0	0	**0**	0.42	0	**0**	0.62	**0.62**
MIMO Size	128×8	128×8	128×16	128×8	**128×8**	128×8	128×8	**128×8**	128×16	4×4	**4×4**	4×4	**4×4**
Frequency [MHz]	205	308	208	258	**800**	300	317	**800**	569	278	**800**	500	**800**
LUTs+FFs	45557	178265	69793	66922	**-**	-	634613	**-**	-	154420	**-**	-	**-**
DSP48	697	1850	544	774	**-**	-	1447	**-**	-	304	**-**	-	**-**
Equivalent Logic Gates[b] [MGE]	8.12	23.50	7.50	9.57	**3.55**	0.148	35.09	**3.55**	3.60	8.08	**3.55**	2.76	**3.55**
Throughput [Gb/s]	0.30	0.73	0.32	0.38	**1.54**	0.3	0.60	**0.88**	1.8	0.51	**0.93**	0.37	**0.97**
Power [W]	-	-	3.17	-	**0.528**	0.018	-	**0.537**	0.127	-	**0.532**	0.549	**0.521**
Energy Efficiency [Gb/s/W]	-	-	0.102	-	**2.917**	16.667	-	**1.639**	14.173	-	**1.748**	0.670	**1.862**
Area Efficiency [Gb/s/MGE]	0.038	0.031	0.043	0.039	**0.434**	2.027	0.017	**0.248**	0.5	0.063	**0.262**	0.133	**0.273**

[a] SNR loss compared with the MMSE algorithm for linear detectors and compared with the SD algorithm for nonlinear detectors (with a frame error rate (FER) target of 10^{-2}).
[b] To achieve fair comparisons between FPGA and CGRA implementations, the FPGA resources are normalized to the number of equivalent logic gates.

TABLE VII
PERFORMANCES OF LUD-BASED MIMO DETECTION

MIMO Size	4×4	8×8	16×16	32×32
Throughput [Gb/s]	1.2	0.95	0.42	0.13
Power [W]	0.545	0.537	0.530	0.525
Area (Logic Gates) [MGE]	3.55	3.55	3.55	3.55
Energy Efficiency [Gb/s/W]	2.202	1.769	0.792	0.247
Area Efficiency [Gb/s/MGE]	0.338	0.268	0.118	0.037

TABLE VIII
PERFORMANCES ON VARIOUS BASEBAND PROCESSING BENCHMARKS

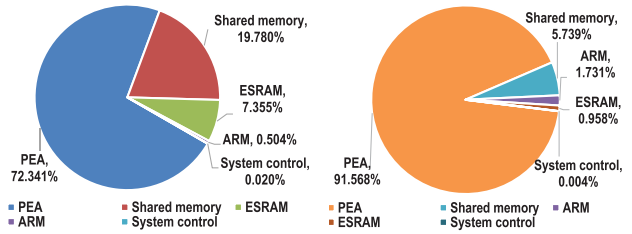
		NCO	HBF	FIR	FFT-1536	FFT-2048
Performance [cycles]	This paper	560	1874	2158	468	652
	TI C66x	6169	44572	43804	5581	7433
Power [mW]	This paper	527	534	514	542	551
	TI C66x	1050	1040	1040	1056	1092

3) Flexibility and Scalability in Terms of Other Baseband Processing Tasks: Although the abovementioned design and experimental results are based on MIMO detection algorithms, the proposed reconfigurable baseband processor is not limited to MIMO detection algorithms. Table VIII shows the implementation results achieved with the proposed reconfigurable baseband processor and the TI C66x DSP [42]–[44] for various baseband processing benchmarks, such as a numerically controlled oscillator (NCO) filter, a half-band filter (HBF), a finite impulse response (FIR) filter, and FFT. For a fair comparison, the data for the DSP were evaluated using only one core because the number of logic gates in one core of the TI C66x is approximately 5 MGE, which is close to that in the proposed architecture (3.55 MGE). Both architectures are implemented in the TSMC 28-nm technology. The frequencies of the proposed architecture and the TI C66x are 800 MHz and 1 GHz, respectively. According to Table VIII, for the NCO, HBF, FIR, FFT-1536, and FFT-2048 implementations, the proposed architecture achieves 27.97× the efficiency (performance/power [1/cycle/mW]) of the TI C66x DSP. The reason for this improvement is that the DSP uses a time-domain computing model, while the proposed CGRA architecture optimizes the algorithm execution in two dimensions: the time domain and the spatial domain. In the time domain, the DSP needs to parse each instruction and execute it. By contrast, the operation of the proposed CGRA is defined in terms of context information. When the configuration process is complete, the CGRA is similar to an ASIC; it can be configured once and then executed many times. In the spatial domain, the DSP mainly uses the SM to reuse data, while the proposed CGRA uses the interconnections between the PEs for explicit message passing. The power consumption of the proposed processor is therefore lower because of the reduced number of memory accesses and the lower communication cost [28]–[30].

B. Analysis of Energy and Area Efficiencies

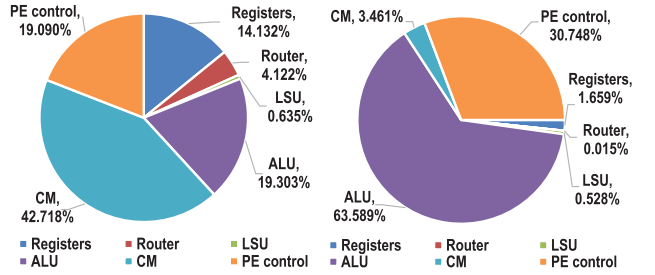
This section focuses on the energy and area efficiencies achieved by the proposed processor on the basis of its flexibility and scalability. Table VI shows the implementation results of the proposed chip and other state-of-the-art designs benchmarked on various algorithms with different MIMO scales. Compared with the CGRA design in [34], the proposed architecture achieves 2.78× and 2.05× higher energy and area efficiencies, respectively. Compared with the FPGA designs in [7], [15]–[17], [23], and [24], the proposed architecture achieves a 28.54× higher energy efficiency and a 4.16–14.43× higher area efficiency. Thus, the proposed architecture has advantages in both energy and area efficiencies. One reason is that the reconfigurable processor architecture proposed in this article is customized for the domain of massive MIMO signal processing, including optimized designs of the data and context paths. Another reason is the optimization techniques, i.e., the OMSA, DMDS, and CACI techniques, which are designed based on the relevant algorithm and architecture characteristics. In an FPGA, a large number of LUTs are used for programming, and flexible crossbars are used for interconnection, thereby ensuring logic and interconnection with single-bit flexibility. This will result in a large amount of context information, high power consumption, and difficulty in achieving high-speed dynamic reconfiguration [25]. Because of the high power consumption, current FPGAs tend to have coarse-grained module designs [25]. The reconfiguration time

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL. Downloaded on June 04,2026 at 03:59:59 UTC from IEEE Xplore. Restrictions apply.



(a) Area Breakdown (b) Power Breakdown

Fig. 15. (a) Area and (b) power breakdowns of the proposed processor.



(a) Area Breakdown (b) Power Breakdown

Fig. 16. (a) Area and (b) power breakdowns of a PE.

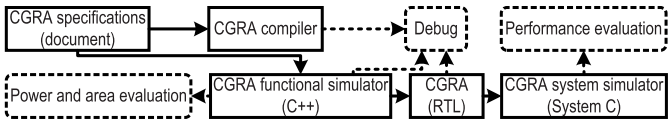


Fig. 17. Design flow of the reconfigurable architecture.

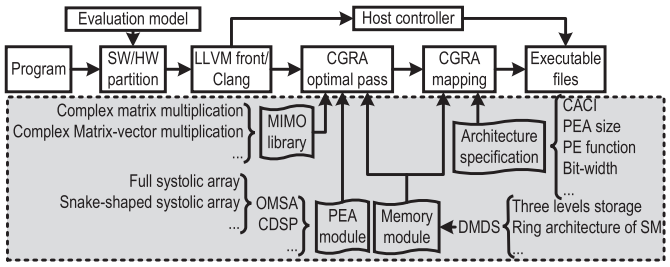


Fig. 18. Compiler design for the proposed architecture.

for run-time reconfiguration and partial reconfiguration of the FPGA is longer than that in the proposed CGRA. Compared with the proposed CGRA, the switching granularity of an FPGA is relatively coarse; consequently, its switching flexibility is insufficient, and switching requires a long time and considerable power [25]–[27]. There are three main types of power consumption in an FPGA: chip-dependent static power consumption, design-dependent static power consumption, and design-dependent dynamic power consumption. Notably, to ensure fair comparisons for the linear algorithms WeJi and CHD, the energy and area efficiencies of the former are compared with those in [15]–[17], [20], and [24], while the efficiencies of the latter are compared with those in [23]. The reason is that the error-rate performances of the WeJi algorithm and the algorithms in [15]–[17], [20], and [24] are similar, while the error-rate performances of the CHD algorithm are similar to that of the algorithm in [23]. In addition, the implementations of the SD and NP algorithms using the proposed architecture are compared with [7] and [34], respectively.

C. Area and Power Breakdowns

The area and power breakdowns of the processor, the PEAs, and the PEs are presented here. The area breakdowns were calculated based on the area report from the place and route tools. The power breakdowns were obtained via post place and route simulations using actual workloads at 800 MHz. As shown in Fig. 15, the PEAs represent a large proportion of the area and power consumption in the processor. In addition, the SM makes a notable contribution to the area and power consumption for the storage of input-output data, parameters, and intermediate data. In a PEA, 96% of the area and 99% of the power are dedicated to the PEs. The area and power breakdowns for a PE are shown in Fig. 16. The ALU, CM, PE controller, and registers, which mainly ensure the flexibility and computational efficiency of the PE, all account for large proportions of the area. The ALU accounts for the largest proportion of the power consumption, indicating that most of the power consumption arises from computation.

VI. Extended Discussion: Tool Chains

Since the tool chains are very important for reconfigurable processors, this extended discussion is presented to briefly introduce the software tool chains, which includes simulator and compiler [45], [46].

A. Simulator

The architecture proposed in this article includes two simulators, namely, a functional simulator and a system simulator, as shown in Fig. 17. The functional simulator can simulate the functions of a PEA, including context package translation and function simulation for a single context package. Based on the specifications, the functional simulator has been built in C++. The system simulator is a cycle-accurate simulator that can simulate the whole hardware system, including PEA, data transfer, context scheduling, and multipackage joint simulations. An elaborate performance report can be extracted from the system simulator. Moreover, power profiling is enabled based on the coprocessor characteristics.

B. Compiler

The compiler (see Fig. 18) is designed for massive MIMO detection and baseband processing. First, the C code is input, and the tasks are divided into hardware and software components. Subsequently, the LLVM front end converts the C code into a DFG. The CGRA optimal pass optimizes the DFG for better task mapping. Then, the DFG is mapped to the CGRA architecture [45], [46]. During this process, special library files can be exploited via template matching, such as a complex matrix multiplication context file in the MIMO library. The PEA module provides the PEA patterns, such as the OMSA configurations (including full systolic arrays and snake-shaped systolic arrays) and the CDSP. The memory module provides the memory architecture patterns, such as the DMDS design (including the three-level storage architecture

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL. Downloaded on June 04,2026 at 03:59:59 UTC from IEEE Xplore. Restrictions apply.

and the ring architecture of SMs). In addition, there are architecture specification selections for the CGRA, which can provide mapping parameters for system elements, such as the CACI, the PEA size, and the PE functions. Finally, the tasks that need to be performed by the host controller and the CGRA are combined to form the context package [47]–[49].

VII. Conclusion

A reconfigurable baseband processor for massive MIMO detection was fabricated in the TSMC 28-nm CMOS technology. To achieve high flexibility, scalability, and energy and area efficiencies for the data path, 4×4 reconfigurable PEAs, domain-customized reconfigurable PEs, simplified reconfigurable interconnections, an optimized SM system, and a hierarchical computation flow are proposed; for the context path, a hierarchical context controller and a prefetched CM system are designed. To flexibly support the requirements of diverse algorithms with high energy and area efficiencies, three optimization techniques are proposed: an OMSA technique based on PEAs, a DMDS technique, and a CACI format. All these optimization techniques improve the flexibility and scalability of the processor. In addition, the implementation results in silicon show that this processor achieves high energy and area efficiencies.

References

[1] T. L. Marzetta, "Noncooperative cellular wireless with unlimited numbers of base station antennas," *IEEE Trans. Wireless Commun.*, vol. 9, no. 11, pp. 3590–3600, Nov. 2010.

[2] F. Rusek *et al.*, "Scaling up MIMO: Opportunities and challenges with very large arrays," *IEEE Signal Process. Mag.*, vol. 30, no. 1, pp. 40–60, Jan. 2013.

[3] H. Q. Ngo, E. G. Larsson, and T. L. Marzetta, "Energy and spectral efficiency of very large multiuser MIMO systems," *IEEE Trans. Commun.*, vol. 61, no. 4, pp. 1436–1449, Apr. 2013.

[4] L. V. der Perre, L. Liu, and E. G. Larsson, "Efficient DSP and circuit architectures for Massive MIMO: State of the art and future directions," *IEEE Trans. Signal Process.*, vol. 66, no. 18, pp. 4717–4736, Sep. 2018.

[5] G. Peng, L. Liu, S. Zhou, Y. Xue, S. Yin, and S. Wei, "Algorithm and architecture of a low-complexity and high-parallelism preprocessing-based k-best detector for large-scale MIMO systems," *IEEE Trans. Signal Process.*, vol. 66, no. 7, pp. 1860–1875, Apr. 2018.

[6] C.-H. Liao, T.-P. Wang, and T.-D. Chiueh, "A 74.8 mW soft-output detector IC for 8×8 spatial-multiplexing MIMO communications," *IEEE J. Solid-State Circuits*, vol. 45, no. 2, pp. 411–421, Feb. 2010.

[7] Y. Wu and J. McAllister, "Architectural synthesis of multi-SIMD dataflow accelerators for FPGA," *IEEE Trans. Parallel Distrib. Syst.*, vol. 29, no. 1, pp. 43–55, Jan. 2018.

[8] W. Tang, H. Prabhu, L. Liu, V. Öwall, and Z. Zhang, "A 1.8 Gb/s 70.6 pJ/b 128×16 link-adaptive near-optimal massive MIMO detector in 28 nm UTBB-FDSOI," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2018, pp. 224–226.

[9] W. Tang, C.-H. Chen, and Z. Zhang, "A 0.58 mm^2 2.76 Gb/s 79.8 pJ/b 256-QAM massive MIMO message-passing detector," in *Proc. IEEE Symp. VLSI Circuits*, Honolulu, HI, USA, Jun. 2016, pp. 1–2.

[10] Y.-T. Chen, C.-C. Cheng, T.-L. Tsai, W.-C. Sun, Y.-L. Ueng, and C.-H. Yang, "A 501 mW 7.6 lGb/s integrated message-passing detector and decoder for polar-coded massive MIMO systems," in *Proc. IEEE Symp. VLSI Circuits*, Kyoto, Japan, Jun. 2017, pp. 330–331.

[11] C. Studer, S. Fateh, and D. Seethaler, "ASIC implementation of soft-input soft-output MIMO detection using MMSE parallel interference cancellation," *IEEE J. Solid-State Circuits*, vol. 46, no. 7, pp. 1754–1765, Jul. 2011.

[12] H. Prabhu, J. N. Rodrigues, L. Liu, and O. Edfors, "A 60 pJ/b 300 Mb/s 128×8 massive MIMO precoder-detector in 28 nm FD-SOI," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2017, pp. 60–61.

[13] C.-H. Chen, W. Tang, and Z. Zhang, "A 2.4mm^2 130 mW MMSE-nonbinary-LDPC iterative detector-decoder for 4×4 256-QAM MIMO in 65 nm CMOS," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2015, pp. 338–340.

[14] B. Yin, M. Wu, G. Wang, C. Dick, J. R. Cavallaro, and C. Studer, "A 3.8 Gb/s large-scale MIMO detector for 3GPP LTE-advanced," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, Florence, Italy, May 2014, pp. 3879–3883.

[15] C. Zhang, Z. Wu, C. Studer, Z. Zhang, and X. You, "Efficient soft-output Gauss-Seidel data detector for massive MIMO systems," *IEEE Trans. Circuits Syst. I, Reg. Papers*, to be published.

[16] O. Castañeda, S. Jacobsson, G. Durisi, M. Coldrey, T. Goldstein, and C. Studer, "1-bit massive MU-MIMO precoding in VLSI," *IEEE J. Emerg. Sel. Topics Circuits Syst.*, vol. 7, no. 4, pp. 508–522, Dec. 2017.

[17] G. Peng, L. Liu, S. Zhou, S. Yin, and S. Wei, "A 1.58 Gbps/W 0.40 Gbps/mm^2 ASIC implementation of MMSE detection for 128×8 64-QAM massive MIMO in 65 nm CMOS," *IEEE Trans. Circuits Syst.*, vol. 65, no. 5, pp. 1717–1730, May 2018.

[18] G. Peng, L. Liu, P. Zhang, S. Yin, and S. Wei, "Low-computing-load, high-parallelism detection method based on Chebyshev iteration for massive MIMO systems with VLSI architecture," *IEEE Trans. Signal Process.*, vol. 65, no. 14, pp. 3775–3788, Jul. 2017.

[19] S. Roger, C. Ramiro, A. Gonzalez, V. Almenar, and A. M. Vidal, "Fully parallel GPU implementation of a fixed-complexity soft-output MIMO detector," *IEEE Trans. Veh. Technol.*, vol. 61, no. 8, pp. 3796–3800, Oct. 2012.

[20] K. Li, R. R. Sharan, Y. Chen, T. Goldstein, J. R. Cavallaro, and C. Studer, "Decentralized baseband processing for massive MU-MIMO systems," *IEEE J. Emerg. Sel. Topics Circuits Syst.*, vol. 7, no. 4, pp. 491–507, Dec. 2017.

[21] J. Chen, Z. Zhang, H. Lu, J. Hu, and G. E. Sobelman, "An intra-iterative interference cancellation detector for large-scale MIMO communications based on convex optimization," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 63, no. 11, pp. 2062–2072, Nov. 2016.

[22] O. Castañeda, T. Goldstein, and C. Studer, "VLSI designs for joint channel estimation and data detection in large SIMO wireless systems," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 65, no. 3, pp. 1120–1132, Mar. 2018.

[23] M. Wu, B. Yin, G. Wang, C. Dick, J. R. Cavallaro, and C. Studer, "Large-scale MIMO detection for 3GPP LTE: Algorithms and FPGA implementations," *IEEE J. Sel. Topics Signal Process.*, vol. 8, no. 5, pp. 916–929, Oct. 2014.

[24] M. Wu, C. Dick, J. R. Cavallaro, and C. Studer, "High-throughput data detection for massive MU-MIMO-OFDM using coordinate descent," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 63, no. 12, pp. 2357–2367, Dec. 2016.

[25] S. M. S. Trimberger, "Three ages of FPGAs: A retrospective on the first thirty years of FPGA technology," *IEEE Solid-State Circuits Mag.*, vol. 10, no. 2, pp. 16–29, Jun. 2018.

[26] K. Compton and S. Hauck, "Reconfigurable computing: A survey of systems and software," *ACM Comput. Surv.*, vol. 34, no. 2, pp. 171–210, 2002.

[27] J. C. Lyke, C. G. Christodoulou, G. A. Vera, and A. H. Edwards, "An introduction to reconfigurable systems," *Proc. IEEE*, vol. 103, no. 3, pp. 291–317, Mar. 2015.

[28] R. Tessier, K. Pocek, and A. DeHon, "Reconfigurable computing architectures," *Proc. IEEE*, vol. 103, no. 3, pp. 332–354, Mar. 2015.

[29] M. Horowitz, "Computing's energy problem (and what we can do about it)," in *Int. Solid-State Circuits Conf. Dig. Tech. Papers (ISSCC)*, Feb. 2014, pp. 10–14.

[30] K. Bondalapati and V. K. Prasanna, "Reconfigurable computing systems," *Proc. IEEE*, vol. 90, no. 7, pp. 1201–1217, Jul. 2002.

[31] C. Yang, L. Liu, K. Luo, S. Yin, and S. Wei, "CIACP: A correlation- and iteration- aware cache partitioning mechanism to improve performance of multiple coarse-grained reconfigurable arrays," *IEEE Trans. Parallel Distrib. Syst.*, vol. 28, no. 1, pp. 29–43, Jan. 2017.

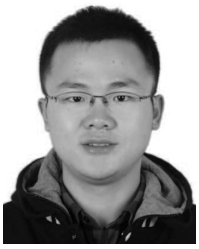
[32] L. Liu *et al.*, "An energy-efficient coarse-grained reconfigurable processing unit for multiple-standard video decoding," *IEEE Trans. Multimedia*, vol. 17, no. 5, pp. 1706–1720, Oct. 2015.

[33] L. Liu, Z. Li, C. Yang, C. Deng, S. Yin, and S. Wei, "HReA: An energy-efficient embedded dynamically reconfigurable fabric for 13-dwarfs processing," *IEEE Trans. Circuits Syst., II, Exp. Briefs*, vol. 65, no. 3, pp. 381–385, Mar. 2018.

[34] C. Zhang, L. Liu, D. Marković, and V. Öwall, "A heterogeneous reconfigurable cell array for MIMO signal processing," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 62, no. 3, pp. 733–742, Mar. 2015.

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL. Downloaded on June 04,2026 at 03:59:59 UTC from IEEE Xplore. Restrictions apply.

[35] C. Zhang, H. Prabhu, Y. Liu, L. Liu, O. Edfors, and V. Öwall, "Energy efficient group-sort QRD processor with on-line update for MIMO channel pre-processing," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 62, no. 5, pp. 1220–1229, May 2015.
[36] X. Chen, A. Minwegen, S. B. Hussain, A. Chattopadhyay, G. Ascheid, and R. Leupers, "Flexible, efficient multimode MIMO detection by using reconfigurable ASIP," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 23, no. 10, pp. 2173–2186, Oct. 2015.
[37] G. Auer *et al.*, "How much energy is needed to run a wireless network?" *IEEE Trans. Wireless Commun.*, vol. 18, no. 5, pp. 40–49, Oct. 2011.
[38] C. Zhang, L. Liu, Y. Wang, M. Zhu, O. Edfors, and V. Owall, "A highly parallelized MIMO detector for vector-based reconfigurable architectures," in *Proc. IEEE Wireless Commun. Netw. Conf. (WCNC)*, Apr. 2013, pp. 3844–3849.
[39] J. M. Rabaey, A. Chandrakasan, and B. Nikolic, *Digital Integrated Circuits*, vol. 2. Englewood Cliffs, NJ, USA: Prentice-Hall, 2002.
[40] *DARPA Rolls Out Electronics Resurgence Initiative*. Accessed: Sep. 13, 2017. [Online]. Available: https://www.darpa.mil/news-events/
[41] S. Nouri, W. Hussain, and J. Nurmi, "Evaluation of a heterogeneous multicore architecture by design and test of an OFDM receiver," *IEEE Trans. Parallel Distrib. Syst.*, vol. 28, no. 11, pp. 3171–3187, Nov. 2017.
[42] *66AK2G12 Multicore DSP+Arm, 1 Arm A15 cores, 1x C66x DSP Core*. Accessed: Jul. 24, 2019. [Online]. Available: http://www.ti.com/product/66AK2G12
[43] *TMS320C6678: Two Test Results for DSPF_sp_fftSPxSP in DSPLIB_C66x_TestReport.html*. Accessed: Jul. 25, 2019. [Online]. Available: http://e2e.ti.com/support/processors/f/791/p/754051/2785557
[44] *66AK2G12 Power Estimation Tool Processors SPRACD9*. Accessed: Jul. 25, 2019. [Online]. Available: http://www.ti.com/analog/docs/litabsmultiplefilelist.tsp?literatureNumber=SPRACD9&docCategoryId=1&familyId=44
[45] S. A. Chin and J. H. Anderson, "An architecture-agnostic integer linear programming approach to CGRA mapping," in *Proc. 56th Annu. ACM/ESDA/IEEE Design Autom. Conf.*, San Francisco, CA, USA, Jun. 2018, pp. 1–6.
[46] S. Windh *et al.*, "High-level language tools for reconfigurable computing," *Proc. IEEE*, vol. 103, no. 3, pp. 390–408, Mar. 2015.
[47] X. Man, L. Liu, J. Zhu, and S. Wei, "A general pattern-based dynamic compilation framework for coarse-grained reconfigurable architectures," in *Proc. 56th Annu. ACM/ESDA/IEEE Design Autom. Conf.*, Las Vegas, NV, USA, Jun. 2019, Art. no. 195.
[48] J. Gu, S. Yin, L. Liu, and S. Wei, "Stress-aware loops mapping on CGRAs with dynamic multi-map reconfiguration," *IEEE Trans. Parallel Distrib. Syst.*, vol. 29, no. 9, pp. 2105–2120, Sep. 2018.
[49] J. Gu, S. Yin, and S. Wei, "Stress-aware loops mapping on CGRAs with considering NBTI aging effect," in *Proc. 54th Annu. ACM/ESDA/IEEE Design Autom. Conf.*, Austin, TX, USA, Jun. 2017, pp. 1–6.



Guiqiang Peng received the B.S. degree from the School of Micro-Electronics and Solid-State Electronics, University of Electronic Science and Technology of China, Chengdu, China, in 2013. He is currently pursuing the Ph.D. degree with the Institute of Microelectronics, Tsinghua University, Beijing, China.

His current research interests include reconfigurable computing, mobile computing, and VLSI signal processing and wireless communications.



Leibo Liu (M'10–SM'16) received the B.S. degree in electronic engineering and the Ph.D. degree from the Institute of Microelectronics, Tsinghua University, Beijing, China, in 1999 and 2004, respectively.

He is currently a Professor with the Institute of Microelectronics, Tsinghua University. His current research interests include reconfigurable computing, mobile computing, and VLSI digital signal processing.



Sheng Zhou (S'06–M'12) received the B.E. and Ph.D. degrees in electronic engineering from Tsinghua University, Beijing, China, in 2005 and 2011, respectively.

He is currently an Associate Professor with the Department of Electronic Engineering, Tsinghua University. His research interests include cross-layer design for multiple-antenna systems, edge computing and caching, and green wireless communications.



Shouyi Yin (M'09) received the B.S., M.S., and Ph.D. degrees in electronic engineering from Tsinghua University, Beijing, China, in 2000, 2002, and 2005, respectively.

He has worked at Imperial College London, South Kensington Campus, London, U.K., as a Research Associate. He is currently an Associate Professor with the Institute of Microelectronics, Tsinghua University. His research interests include mobile computing, wireless communications, and system on chip (SoC) design.



Shaojun Wei (M'91–F'19) was born in Beijing, China, in 1958. He received the Ph.D. degree from the La Faculté Polytechnique de Mons, Mons, Belgium, in 1991.

He became a Professor at the Institute of Microelectronics, Tsinghua University, in 1995. His main research interests include VLSI system on chip (SoC) design, electronics design automation (EDA) methodology, and application-specific integrated circuit (ASIC) design for communications.

Dr. Wei is a Senior Member of the Chinese Institute of Electronics (CIE).

Authorized licensed use limited to: UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL. Downloaded on June 04,2026 at 03:59:59 UTC from IEEE Xplore. Restrictions apply.